SISTEMA DE GESTÃO DE LIVRARIA

Descrição do Sistema

O Sistema de Gestão de Livraria tem como objetivo gerenciar as operações de uma livraria, incluindo cadastro de clientes, controle de livros em estoque, registro de vendas e geração de relatórios.

O sistema busca automatizar processos, reduzir erros e melhorar a organização do negócio, permitindo controle eficiente de estoque e vendas, algo essencial para livrarias modernas.

Requisitos Funcionais

Requisitos funcionais definem o que o sistema deve fazer, ou seja, suas funcionalidades principais.

❖ Gestão de Clientes

- Cadastrar cliente
- Editar cliente
- Listar clientes
- Buscar cliente por ID

📌 **Regras:**

- CPF não pode ser duplicado
- Email deve ser válido
- Cliente não pode ser removido se possuir vendas registradas

❖ Gestão de Livros (Produtos)

- Cadastrar livro
- Listar livros
- Editar livro
- Controlar estoque

📌 **Regras:**

- Preço não pode ser negativo
- Estoque mínimo deve ser controlado
- Não permitir venda se estoque insuficiente

❖ Registro de Vendas

- Registrar venda
- Associar cliente à venda
- Adicionar lista de livros
- Calcular valor total automaticamente
- Registrar usuário responsável

📌 **Regras:**

- Não permitir venda sem itens
- Atualizar estoque após venda
- Não permitir venda com estoque insuficiente

❖ Autenticação

- Login com usuário e senha
- Geração de token JWT
- Controle de acesso por perfil:
    - ADMIN
    - FUNCIONARIO

❖ Relatórios

- Vendas por período
- Vendas por cliente
- Vendas por produto

📌 **Regras:**

- Informar as vendas por período
- Informar vendas do cliente
- Construir representação gráfica das vendas anuais

Requisitos Não Funcionais

- Sistema baseado em arquitetura em camadas
- API REST com retorno em JSON
- Sistema web responsivo
- Uso de banco de dados relacional
- Segurança com autenticação via token JWT
- Código organizado
- Boa performance e escalabilidade
- Facilidade de manutenção

Arquitetura do Sistema

O sistema segue o padrão de arquitetura em camadas, dividido em:

- Camada de Apresentação (Interface)

Responsável pela interação com o usuário

- Camada de Aplicação (API REST)

Responsável por receber e processar requisições

- Camada de Negócio (Service)

Responsável pelas regras de negócio

- Camada de Dados (Repository)

Responsável pelo acesso ao banco de dados

Essa separação facilita manutenção, organização e escalabilidade do sistema.

Padrões de Projeto Utilizados

- MVC (Model-View-Controller)

Separa interface, lógica e dados

- Repository Pattern

Centraliza o acesso ao banco de dados

- DTO (Data Transfer Object)

Utilizado para transportar dados entre camadas

- REST

Comunicação entre cliente e servidor via HTTP

Segurança

O sistema utiliza autenticação baseada em token JWT, garantindo que apenas usuários autenticados possam acessar os recursos da aplicação.

• Endpoint de login para geração de token

• Proteção de rotas da API

• Controle de acesso baseado em perfil:

- ADMIN (acesso total)
- FUNCIONARIO (acesso limitado)

API REST

Principais endpoints:

🔐 Autenticação

• POST /auth/login

👤 Clientes

• GET /clientes

• POST /clientes

• PUT /clientes/{id}

• DELETE /clientes/{id}

📚 Livros

• GET /livros

• POST /livros

• PUT /livros/{id}

• DELETE /livros/{id}

💰 Vendas

• GET /vendas

• POST /vendas

Todas as respostas são retornadas no formato JSON.

Banco de Dados

O sistema utiliza um banco de dados relacional com as seguintes tabelas:

• usuarios

• clientes

• livros

• vendas

• itens_venda

Diagrama de Classe

O diagrama de classes mostra a estrutura do sistema com classes, atributos e relações.

Classe Cliente

- id: int

- nome: String

- cpf: String

- email: String

- telefone: String

- endereco: String

Classe Livro

- id: int

- titulo: String

- descricao: String

- preco: double

- quantidade: int

Classe Usuario

- id: int

- username: String

- senha: String

- perfil: String

Classe Venda

- id: int

- data: Date

- valorTotal: double

-lista de itens: String

- usuário responsável: String

Classe ItemVenda

- id: int

- quantidade: int

Relacionamentos:

• Cliente → Vendas (1:N)

• Venda → Itens de Venda (1:N)

• Livro → Itens de Venda (1:N)

As tabelas possuem relacionamentos através de chaves primárias (PK) e estrangeiras (FK), garantindo integridade dos dados.

Exemplo de relacionamentos:

• Cliente → Vendas (1:N)

• Venda → Itens de Venda (1:N)

• Livro → Itens de Venda (1:N)

Diagrama de Domínio

Cliente ---- realiza ---- Venda

Venda ---- contém ---- Livro

Usuario ---- registra ---- Venda

Diagrama Lógico do Banco (DER)

Baseado no modelo de livraria:

CLIENTES (id, nome, cpf, email, telefone, endereco)

LIVROS (id, titulo, descricao, preco, quantidade)

USUARIOS (id, username, senha, perfil)

VENDAS (id, data, cliente_id, usuario_id, valor_total)

ITENS_VENDA (id, venda_id, livro_id, quantidade)

📌 Relacionamentos:

Cliente → Vendas (1:N)

Venda → Itens (1:N)

Livro → Itens (1:N)

Imagem do diagrama anexado no GIT

Cadastro de Clientes e Produtos

Cliente

Campos:

- nome
- cpf
- email
- telefone
- endereço

Regras:

- CPF único
- Não excluir com venda

Livro

Campos:

- título
- descrição
- preço
- quantidade

Regras:

- Preço ≥ 0
- Estoque ≥ 0

Como Executar o Projeto

1. Instalar as dependências do projeto
2. Configurar o banco de dados MySQL
3. Executar o script de criação das tabelas
4. Iniciar o servidor da aplicação

5. Acessar o sistema via navegador

Link do repositório GitHub

https://github.com/susanabsilva/sistema-livraria-.git